PYTHON PROGRAMMING ASSIGNMENT

10 Case Studies with Complete Beginner-Level Solutions

B.Tech First Year

Topics: Problem Statement, Objectives, Assumptions, Algorithm, Flowchart, Pseudocode, Python Source Code, Test Cases, Sample Output, and Conclusion

Prepared using basic Python concepts

CASE STUDY 1: UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

To develop a Python program to check students' eligibility for scholarships based on aggregate marks, minimum subject marks, attendance, and family income. The program ranks eligible students according to their marks.

2. Objectives

• To check scholarship eligibility.

• To use conditional statements to apply eligibility rules.

• To store eligible students using lists and tuples.

• To arrange students according to their marks.

• To display the final scholarship ranking.

3. Assumptions

• Minimum aggregate marks required are 75%.

• Minimum subject marks required are 60%.

• Minimum attendance required is 75%.

• Family income must not exceed ■2,00,000.

• Students with higher aggregate marks receive higher ranks.

4. Algorithm

Start the program.

Read the number of students.

Enter each student's name, aggregate marks, subject marks, attendance, and family income.

Check all eligibility conditions.

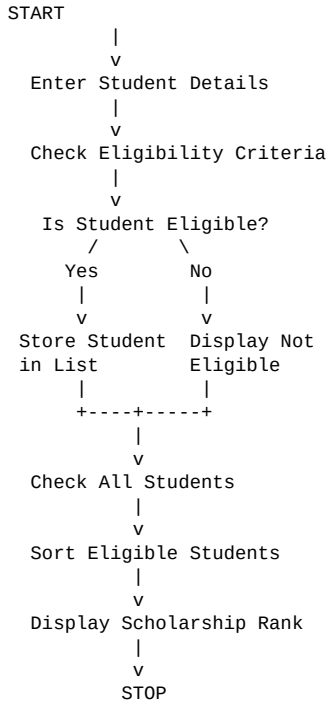
Store eligible students in a list.

Sort eligible students by aggregate marks in descending order.

Display the eligible students and their ranks.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT number of students

FOR each student
    INPUT name, marks, subject marks
    INPUT attendance, family income

    IF marks >= 75 AND subject marks >= 60
       AND attendance >= 75 AND income <= 200000
        STORE student details
    ELSE
        DISPLAY "Not Eligible"
    END IF
END FOR

SORT eligible students by marks
DISPLAY student ranking

STOP
```

7. Python Source Code

```
n = int(input("Enter number of students: "))
students = []

for i in range(n):
    name = input("Enter name: ")
    marks = float(input("Enter aggregate marks: "))
    subject = float(input("Enter subject marks: "))
    attendance = float(input("Enter attendance: "))
    income = float(input("Enter family income: "))

    if marks >= 75 and subject >= 60 and attendance >= 75 and income <= 200000:
        students.append((name, marks))
        print("Eligible")
    else:
        print("Not Eligible")

for i in range(len(students)):
    for j in range(i + 1, len(students)):
        if students[i][1] < students[j][1]:
            temp = students[i]
            students[i] = students[j]
            students[j] = temp

print("Scholarship Ranking")

for i in range(len(students)):
    print(i + 1, students[i][0], students[i][1])
```

8. Test Cases

Test Case	Marks	Subject Marks	Attendance	Income	Expected Result
1	85	70	90	150000	Eligible
2	65	70	90	150000	Not Eligible
3	90	80	60	150000	Not Eligible
4	80	65	85	250000	Not Eligible

9. Sample Output

```
Enter number of students: 2

Enter name: Ravi
Enter aggregate marks: 85
Enter subject marks: 70
Enter attendance: 90
Enter family income: 150000
Eligible

Enter name: Sita
Enter aggregate marks: 92
Enter subject marks: 80
Enter attendance: 95
Enter family income: 180000
Eligible

Scholarship Ranking
1 Sita 92.0
2 Ravi 85.0
```

10. Conclusion

The program checks scholarship eligibility using conditional statements and ranks eligible students according to their aggregate marks. It demonstrates the use of variables, lists, tuples, loops, and sorting.

CASE STUDY 2: PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

To develop a Python program to calculate electricity bills based on the number of units consumed. The program applies different rates for different consumption slabs, adds fixed charges, applies rebates, and compares monthly bills.

2. Objectives

• To calculate electricity charges.

• To apply different consumption slabs.

• To add fixed charges and rebates.

• To store monthly bills in a list.

• To identify unusual increases in bills.

3. Assumptions

• First 100 units cost ■2 per unit.

• Units from 101 to 200 cost ■3 per unit.

• Units from 201 to 300 cost ■5 per unit.

• Units above 300 cost ■7 per unit.

• Fixed charge is ■50.

• A rebate of ■100 is provided if the bill exceeds ■1,000.

• An increase greater than 20% is considered unusual.

4. Algorithm

Start the program.

Create an empty list for monthly bills.

Read electricity units for three months.

Calculate the bill according to the applicable slab.

Add the fixed charge.

Apply the rebate if applicable.

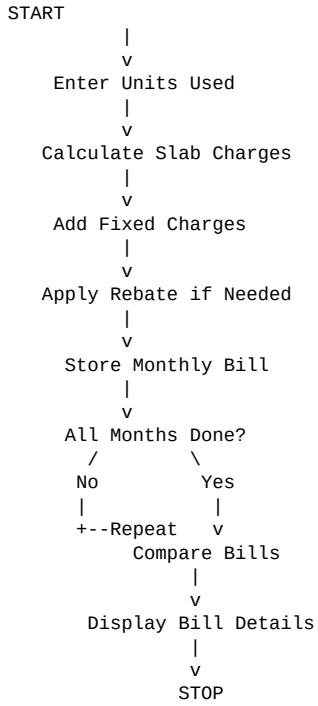
Store the bill in the list.

Compare consecutive monthly bills.

Display unusual increases.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
CREATE empty list bills

REPEAT 3 times
    INPUT units

    IF units <= 100
        bill = units * 2
    ELSE IF units <= 200
        bill = 200 + (units - 100) * 3
    ELSE IF units <= 300
        bill = 500 + (units - 200) * 5
    ELSE
        bill = 1000 + (units - 300) * 7
    END IF

    bill = bill + 50

    IF bill > 1000
        bill = bill - 100
    END IF

    ADD bill to bills
END REPEAT

COMPARE consecutive bills
DISPLAY monthly bills

STOP
```

7. Python Source Code

```
bills = []

for i in range(3):
    units = int(input("Enter electricity units: "))

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 100 * 2 + (units - 100) * 3
    elif units <= 300:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5
    else:
        bill = 100 * 2 + 100 * 3 + 100 * 5 + (units - 300) * 7

    bill = bill + 50

    if bill > 1000:
        bill = bill - 100

    bills.append(bill)
    print("Electricity bill:", bill)

for i in range(1, 3):
    if bills[i] > bills[i - 1] * 1.2:
        print("Unusual increase in month", i + 1)
```

8. Test Cases

Test Case	Units	Expected Result
1	100	■250
2	150	■400
3	250	■900
4	350	■1,600

9. Sample Output

```
Enter electricity units: 100
Electricity bill: 250

Enter electricity units: 150
Electricity bill: 400

Enter electricity units: 250
Electricity bill: 900

Unusual increase in month 3
```

10. Conclusion

The program calculates electricity bills using consumption slabs, fixed charges, and rebates. It also compares monthly bills and identifies unusual increases using loops, lists, and conditional statements.

CASE STUDY 3: HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

To develop a Python program to calculate hospital bills using consultation, diagnostic, and treatment charges. The program applies concessions based on age and classifies patients according to their payment status.

2. Objectives

• To calculate the total hospital bill.

• To apply age-based concessions.

• To validate billing details.

• To calculate the balance amount.

• To classify patients based on payment status.

3. Assumptions

• Patients aged 60 years or above receive a 10% concession.

• All charges must be non-negative.

• Patients who pay the full amount are classified as fully paid.

• Patients who pay some amount are classified as partially paid.

• Patients who pay nothing are classified as not paid.

4. Algorithm

Start the program.

Read patient details and age.

Read consultation, diagnostic, and treatment charges.

Read the amount paid.

Validate the input values.

Calculate the total bill.

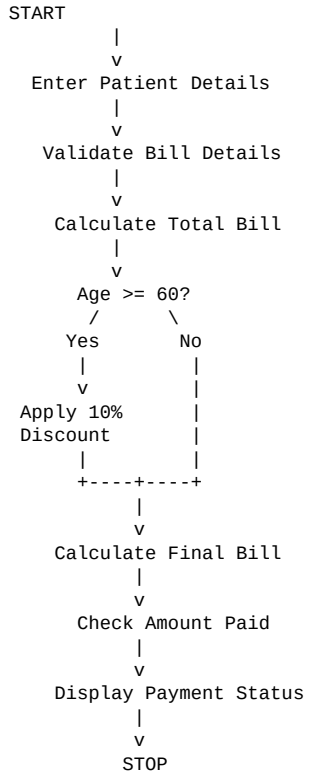
Apply the concession if applicable.

Calculate the final bill and balance.

Display the payment status.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT patient name and age
INPUT consultation charges
INPUT diagnostic charges
INPUT treatment charges
INPUT amount paid

IF any charge or payment is negative
    DISPLAY "Invalid billing details"
ELSE
    total = consultation + diagnostic + treatment

    IF age >= 60
        discount = total * 0.10
    ELSE
        discount = 0
    END IF

    final_bill = total - discount
    balance = final_bill - amount paid

    IF amount paid >= final_bill
        DISPLAY "Fully Paid"
    ELSE IF amount paid > 0
        DISPLAY "Partially Paid"
    ELSE
        DISPLAY "Not Paid"
    END IF
END IF

STOP
```

7. Python Source Code

```
name = input("Enter patient name: ")
age = int(input("Enter patient age: "))

consultation = float(input("Enter consultation charges: "))
diagnostic = float(input("Enter diagnostic charges: "))
treatment = float(input("Enter treatment charges: "))
paid = float(input("Enter amount paid: "))

if consultation < 0 or diagnostic < 0 or treatment < 0 or paid < 0:
    print("Invalid billing details")
else:
    total = consultation + diagnostic + treatment

    if age >= 60:
        discount = total * 0.10
    else:
        discount = 0

    final_bill = total - discount
    balance = final_bill - paid

    if paid >= final_bill:
        status = "Fully Paid"
    elif paid > 0:
        status = "Partially Paid"
    else:
        status = "Not Paid"

    print("Patient Name:", name)
    print("Total Bill:", total)
    print("Discount:", discount)
    print("Final Bill:", final_bill)
    print("Payment Status:", status)
    print("Balance:", balance)
```

8. Test Cases

Test Case	Age	Total Charges	Amount Paid	Expected Result
1	65	■4,000	■3,600	Fully Paid
2	30	■2,000	■1,000	Partially Paid
3	25	■1,500	■0	Not Paid
4	40	Negative	■500	Invalid Details

9. Sample Output

```
Enter patient name: Ravi
Enter patient age: 65
Enter consultation charges: 500
Enter diagnostic charges: 1000
Enter treatment charges: 2500
Enter amount paid: 2000

Patient Name: Ravi
Total Bill: 4000.0
Discount: 400.0
Final Bill: 3600.0
Payment Status: Partially Paid
Balance: 1600.0
```

10. Conclusion

The program calculates hospital bills, applies age-based concessions, and identifies payment status. It demonstrates basic calculations, conditional statements, and input validation.

CASE STUDY 4: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

To develop a Python program to calculate employee salaries using basic pay, allowances, overtime, and incentives. The program calculates deductions and displays the final salary statement.

2. Objectives

• To calculate gross salary.

• To apply salary-based deductions.

• To calculate net salary.

• To display an employee salary statement.

• To understand arithmetic operations and conditional statements.

3. Assumptions

• Gross salary is the sum of basic pay, allowances, overtime, and incentives.

• Salary up to ■20,000 has a 5% deduction.

• Salary from ■20,001 to ■40,000 has a 10% deduction.

• Salary above ■40,000 has a 15% deduction.

• All salary inputs are non-negative.

4. Algorithm

Start the program.

Read the employee's name.

Read basic pay, allowances, overtime, and incentives.

Calculate gross salary.

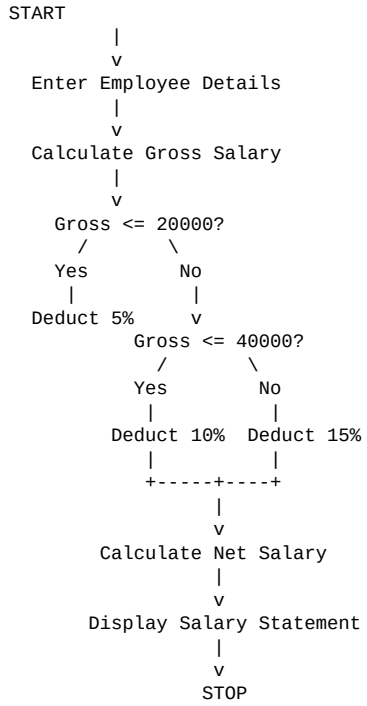
Calculate deductions according to the salary.

Calculate net salary.

Display the salary statement.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT employee name
INPUT basic pay, allowances, overtime, incentives

gross = basic + allowances + overtime + incentives

IF gross <= 20000
    deduction = gross * 0.05
ELSE IF gross <= 40000
    deduction = gross * 0.10
ELSE
    deduction = gross * 0.15
END IF

net_salary = gross - deduction

DISPLAY gross, deduction, net_salary
STOP
```

7. Python Source Code

```
name = input("Enter employee name: ")

basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowances: "))
overtime = float(input("Enter overtime amount: "))
incentive = float(input("Enter incentive: "))

gross = basic + allowance + overtime + incentive

if gross <= 20000:
    deduction = gross * 0.05
elif gross <= 40000:
    deduction = gross * 0.10
else:
    deduction = gross * 0.15

net_salary = gross - deduction

print("Employee Name:", name)
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net_salary)
```

8. Test Cases

Test Case	Gross Salary	Deduction Rate	Expected Deduction
1	■15,000	5%	■750
2	■28,000	10%	■2,800
3	■50,000	15%	■7,500

9. Sample Output

```
Enter employee name: Ravi
Enter basic salary: 20000
Enter allowances: 5000
Enter overtime amount: 2000
Enter incentive: 1000

Employee Name: Ravi
Gross Salary: 28000.0
Deduction: 2800.0
Net Salary: 25200.0
```

10. Conclusion

The program calculates gross salary, deductions, and net salary using arithmetic operators and conditional statements. It helps students understand a basic payroll system.

CASE STUDY 5: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. Problem Statement

To develop a Python program to compare different mobile plans based on data usage and budget. The program calculates extra charges and selects the cheapest eligible plan.

2. Objectives

• To compare multiple mobile plans.

• To calculate additional data charges.

• To check budget restrictions.

• To identify the cheapest suitable plan.

• To use lists, tuples, and loops.

3. Assumptions

• Plan A provides 10 GB for ■199.

• Plan B provides 20 GB for ■299.

• Plan C provides 30 GB for ■399.

• Additional data charges depend on the selected plan.

• The cheapest plan within the budget is selected.

4. Algorithm

Start the program.

Read monthly data usage and budget.

Store mobile plan details in a list of tuples.

Calculate extra charges for each plan.

Calculate the total cost of each plan.

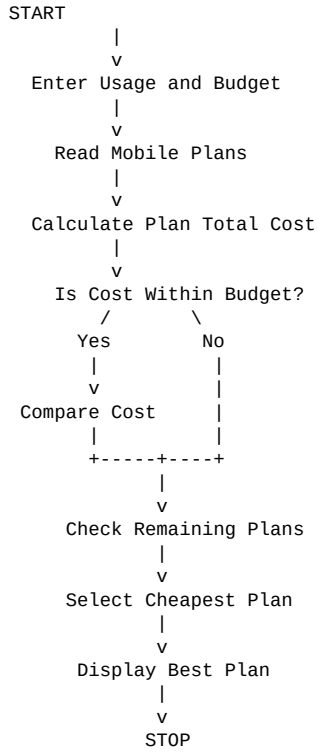
Check whether the cost is within the budget.

Select the cheapest eligible plan.

Display the selected plan and cost.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT usage and budget

STORE mobile plans in a list
best_plan = empty
lowest_cost = large value

FOR each plan
    CALCULATE extra charges
    CALCULATE total cost

    IF total cost <= budget AND total cost < lowest_cost
        lowest_cost = total cost
        best_plan = current plan
    END IF
END FOR

IF best_plan is not empty
    DISPLAY best_plan and lowest_cost
ELSE
    DISPLAY "No suitable plan"
END IF

STOP
```

7. Python Source Code

```
usage = int(input("Enter monthly data usage in GB: "))
budget = float(input("Enter your budget: "))

plans = [
    ("Plan A", 10, 199, 20),
    ("Plan B", 20, 299, 15),
    ("Plan C", 30, 399, 10)
]

best_plan = ""
lowest_cost = 999999

for plan in plans:
    name, data, price, extra_rate = plan

    if usage > data:
        extra = (usage - data) * extra_rate
    else:
        extra = 0

    total = price + extra

    print(name, "Total Cost:", total)

    if total <= budget and total < lowest_cost:
        lowest_cost = total
        best_plan = name

if best_plan != "":
    print("Best Plan:", best_plan)
    print("Total Cost:", lowest_cost)
else:
    print("No plan within budget")
```

8. Test Cases

Test Case	Usage	Budget	Expected Result
1	15 GB	■400	Plan A
2	25 GB	■500	Plan B
3	5 GB	■150	No suitable plan

9. Sample Output

```
Enter monthly data usage in GB: 15
Enter your budget: 400

Plan A Total Cost: 299
Plan B Total Cost: 299
Plan C Total Cost: 399
Best Plan: Plan A
Total Cost: 299
```

10. Conclusion

The program compares mobile plans and identifies the cheapest plan within the given budget. It demonstrates the use of tuples, lists, loops, and conditional statements.

CASE STUDY 6: RAILWAY FARE AND RESERVATION CALCULATOR

1. Problem Statement

To develop a Python program to calculate railway fares based on travel distance, class, and passenger age. The program also calculates reservation fees and cancellation penalties.

2. Objectives

• To calculate railway fares.

• To apply distance-based rates.

• To apply age-based concessions.

• To calculate reservation charges.

• To calculate cancellation penalties and refunds.

3. Assumptions

• Distance up to 100 km has a rate of ■1 per km.

• Distance from 101 to 500 km has a rate of ■1.50 per km.

• Distance above 500 km has a rate of ■2 per km.

• Sleeper reservation fee is ■20.

• AC reservation fee is ■50.

• Passengers below 5 years receive a zero base fare.

• Passengers aged 60 or above receive a 50% concession on the base fare.

• Cancellation penalty is 20% of the total fare.

4. Algorithm

Start the program.

Read the number of passengers.

Enter each passenger's name, age, distance, and class.

Calculate the distance-based fare.

Apply the class rate.

Apply the age concession.

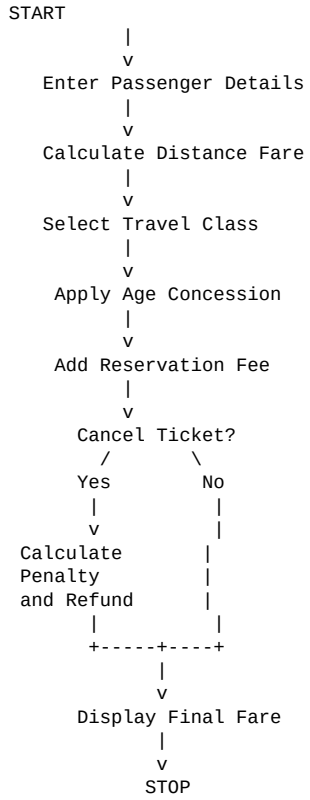
Add the reservation fee.

Ask whether the ticket is cancelled.

If cancelled, calculate the penalty and refund.

Display the fare and stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT number of passengers

FOR each passenger
    INPUT name, age, distance, class

    IF distance <= 100
        rate = 1
    ELSE IF distance <= 500
        rate = 1.5
    ELSE
        rate = 2
    END IF

    CALCULATE fare based on distance and class

    IF age < 5
        fare = 0
    ELSE IF age >= 60
        fare = fare * 0.50
    END IF

    ADD reservation fee

    INPUT cancellation choice

    IF ticket is cancelled
        penalty = total * 0.20
        refund = total - penalty
        DISPLAY penalty and refund
    END IF
END FOR

STOP
```

7. Python Source Code

```
n = int(input("Enter number of passengers: "))

for i in range(n):
    name = input("Enter passenger name: ")
    age = int(input("Enter age: "))
    distance = int(input("Enter distance in km: "))

    print("1. Sleeper")
    print("2. AC")
    choice = int(input("Select class: "))

    if distance <= 100:
        rate = 1
    elif distance <= 500:
        rate = 1.5
    else:
        rate = 2

    if choice == 1:
        class_rate = rate
        reservation = 20
    elif choice == 2:
        class_rate = rate * 2
        reservation = 50
    else:
        print("Invalid class")
        continue

    fare = distance * class_rate

    if age < 5:
        fare = 0
    elif age >= 60:
        fare = fare * 0.50

    total = fare + reservation

    print("Passenger Name:", name)
    print("Total Fare:", total)

    cancel = input("Cancel ticket? (yes/no): ")

    if cancel == "yes":
        penalty = total * 0.20
        refund = total - penalty
        print("Penalty:", penalty)
        print("Refund:", refund)
```

8. Test Cases

Test Case	Age	Distance	Class	Expected Result
1	30	100	Sleeper	■120
2	65	200	Sleeper	■170
3	40	600	AC	■2,450
4	4	100	Sleeper	■20 reservation fee

9. Sample Output

```
Enter number of passengers: 1
Enter passenger name: Ravi
Enter age: 65
Enter distance in km: 200
1. Sleeper
2. AC
Select class: 1

Passenger Name: Ravi
Total Fare: 170.0
Cancel ticket? (yes/no): yes
Penalty: 34.0
Refund: 136.0
```

10. Conclusion

The program calculates railway fares using distance, class, and age. It also demonstrates reservation charges, cancellation penalties, and refunds using conditional statements.

CASE STUDY 7: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

To develop a Python program to generate a retail invoice for multiple products. The program calculates product amounts, applies discounts, calculates category-based taxes, and displays the final bill.

2. Objectives

• To calculate product costs.

• To apply discounts based on purchase amount.

• To calculate category-based taxes.

• To generate an itemized invoice.

• To calculate the total retail bill.

3. Assumptions

• Purchases of ■5,000 or more receive a 10% discount.

• Purchases of ■2,000 or more receive a 5% discount.

• Other purchases receive no discount.

• Food products have an assumed tax rate of 5%.

• Electronics have an assumed tax rate of 18%.

• Tax is calculated after the discount.

4. Algorithm

Start the program.

Read the number of products.

Enter product name, price, quantity, and category.

Calculate the product amount.

Apply the discount.

Calculate tax based on the category.

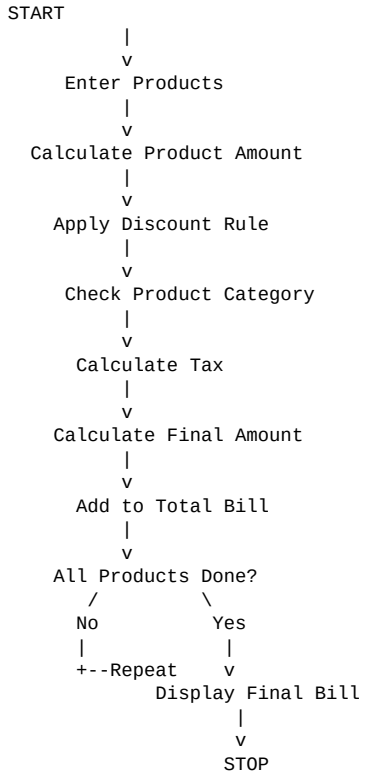
Calculate the final amount.

Add each product's final amount to the total bill.

Display the invoice.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT number of products
total = 0

FOR each product
    INPUT name, price, quantity, category

    amount = price * quantity

    IF amount >= 5000
        discount = amount * 0.10
    ELSE IF amount >= 2000
        discount = amount * 0.05
    ELSE
        discount = 0
    END IF

    IF category = "food"
        tax_rate = 0.05
    ELSE
        tax_rate = 0.18
    END IF

    discounted_amount = amount - discount
    tax = discounted_amount * tax_rate
    final_amount = discounted_amount + tax

    total = total + final_amount
    DISPLAY product details
END FOR

DISPLAY total bill
STOP
```

7. Python Source Code

```
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    name = input("Enter product name: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: "))
    category = input("Enter category (food/electronics): ")

    amount = price * quantity

    if amount >= 5000:
        discount = amount * 0.10
    elif amount >= 2000:
        discount = amount * 0.05
    else:
        discount = 0

    if category == "food":
        tax_rate = 0.05
    else:
        tax_rate = 0.18

    discounted_amount = amount - discount
    tax = discounted_amount * tax_rate
    final_amount = discounted_amount + tax

    print("Product:", name)
    print("Amount:", amount)
    print("Discount:", discount)
    print("Tax:", tax)
    print("Final Amount:", final_amount)

    total = total + final_amount

print("Total Retail Bill:", total)
```

8. Test Cases

Test Case	Price	Quantity	Category	Expected Final Amount
1	■1,000	1	Food	■1,050
2	■1,000	2	Food	■1,995
3	■3,000	2	Electronics	■6,372

9. Sample Output

```
Enter number of products: 1
Enter product name: Headphones
Enter price: 3000
Enter quantity: 2
Enter category (food/electronics): electronics

Product: Headphones
Amount: 6000.0
Discount: 600.0
Tax: 972.0
Final Amount: 6372.0

Total Retail Bill: 6372.0
```

10. Conclusion

The program generates a retail bill by calculating product amounts, discounts, and taxes. It demonstrates loops, conditional statements, and arithmetic operations.

CASE STUDY 8: MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. Problem Statement

To develop a Python program to calculate manufacturing costs, including material, wastage, labour, and machine costs. The program determines whether the production meets the target profit.

2. Objectives

• To calculate material costs.

• To calculate wastage costs.

• To calculate labour and machine costs.

• To calculate total revenue and profit.

• To check whether the target profit is achieved.

3. Assumptions

• Wastage cost is 5% of the material cost.

• Material and labour costs are calculated per product.

• Machine cost is a fixed production expense.

• Profit equals total revenue minus total cost.

• The target profit is provided by the user.

4. Algorithm

Start the program.

Read the production quantity.

Read material cost, labour cost, machine cost, and selling price.

Calculate total material cost.

Calculate wastage and labour costs.

Calculate total manufacturing cost.

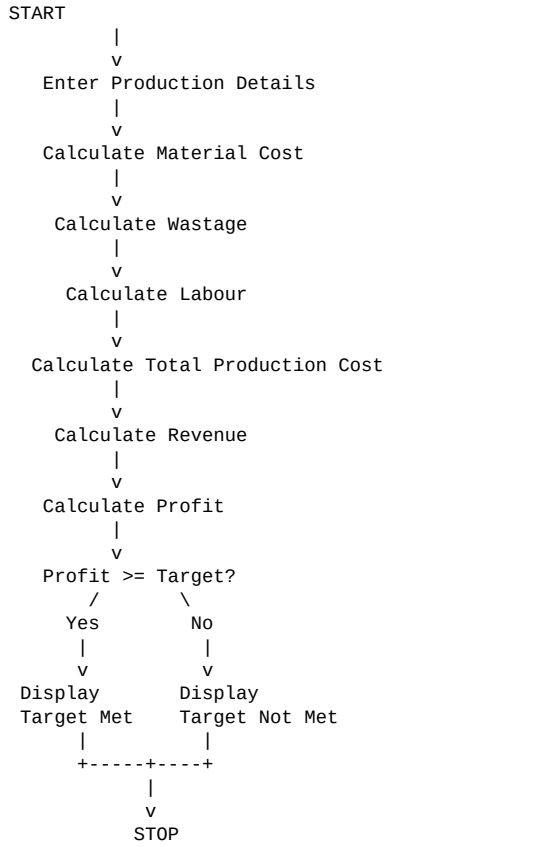
Calculate total revenue.

Calculate profit.

Compare profit with the target.

Display the result and stop.

5. Flowchart



6. Pseudocode

```
START
INPUT quantity
INPUT material cost per product
INPUT labour cost per product
INPUT machine cost
INPUT selling price per product
INPUT target profit

material_total = material * quantity
wastage = material_total * 0.05
labour_total = labour * quantity

total_cost = material_total + wastage
total_cost = total_cost + labour_total + machine

revenue = selling_price * quantity
profit = revenue - total_cost

IF profit >= target profit
    DISPLAY "Target profit achieved"
ELSE
    DISPLAY "Target profit not achieved"
END IF

STOP
```

7. Python Source Code

```
quantity = int(input("Enter number of products: "))
material = float(input("Enter material cost per product: "))
labour = float(input("Enter labour cost per product: "))
machine = float(input("Enter machine cost: "))
selling_price = float(input("Enter selling price per product: "))
target = float(input("Enter target profit: "))

material_total = material * quantity
wastage = material_total * 0.05
labour_total = labour * quantity

total_cost = material_total + wastage + labour_total + machine
revenue = selling_price * quantity
profit = revenue - total_cost

print("Material Cost:", material_total)
print("Wastage Cost:", wastage)
print("Labour Cost:", labour_total)
print("Machine Cost:", machine)
print("Total Cost:", total_cost)
print("Revenue:", revenue)
print("Profit:", profit)

if profit >= target:
    print("Target profit achieved")
else:
    print("Target profit not achieved")
```

8. Test Cases

Test Case	Quantity	Selling Price	Target Profit	Expected Result
1	100	■100	■1,000	Target achieved
2	100	■100	■2,000	Target not achieved
3	50	■200	■1,000	Depends on total cost

9. Sample Output

```
Enter number of products: 100
Enter material cost per product: 50
Enter labour cost per product: 20
Enter machine cost: 1000
Enter selling price per product: 100
Enter target profit: 2000

Material Cost: 5000.0
Wastage Cost: 250.0
Labour Cost: 2000.0
Machine Cost: 1000.0
Total Cost: 8250.0
Revenue: 10000.0
Profit: 1750.0
Target profit not achieved
```

10. Conclusion

The program calculates manufacturing costs and profit. It helps determine whether the production process meets a target profit using basic arithmetic and conditional statements.

CASE STUDY 9: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. Problem Statement

To develop a Python program to calculate travel expenses in Indian rupees using foreign currency exchange rates. The program includes conversion fees, compares spending with a budget, and identifies the largest expense category.

2. Objectives

• To convert foreign currency into Indian rupees.

• To calculate currency conversion fees.

• To calculate total travel expenses.

• To compare expenses against the budget.

• To identify the largest expense category.

3. Assumptions

• Travel expenses are divided into hotel, food, and transport.

• Exchange rates are provided by the user.

• The conversion fee is 2% of the converted amount.

• The total expenses are compared against the travel budget.

• The largest expense is identified by comparing category totals.

4. Algorithm

Start the program.

Read the travel budget.

Create a list of expense categories.

Enter the expense and exchange rate for each category.

Convert each expense into Indian rupees.

Add the conversion fee.

Store the converted expenses in a list.

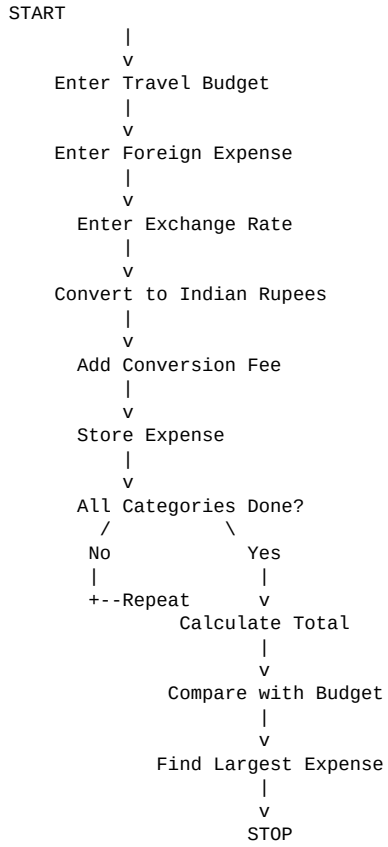
Calculate total expenses.

Compare expenses with the budget.

Identify the largest expense category.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT budget

CREATE categories list
CREATE empty expenses list

FOR each category
    INPUT foreign amount
    INPUT exchange rate

    converted = amount * rate
    fee = converted * 0.02
    total = converted + fee

    ADD total to expenses
END FOR

CALCULATE total expenses

IF total expenses <= budget
    DISPLAY "Within budget"
ELSE
    DISPLAY "Budget exceeded"
END IF

FIND largest expense
DISPLAY largest category

STOP
```

7. Python Source Code

```
budget = float(input("Enter travel budget in INR: "))

categories = ["Hotel", "Food", "Transport"]
expenses = []

for i in range(3):
    print(categories[i])

    amount = float(input("Enter foreign currency amount: "))
    rate = float(input("Enter exchange rate: "))

    converted = amount * rate
    fee = converted * 0.02
    total = converted + fee

    expenses.append(total)

    print("Expense in INR:", total)

total_expense = sum(expenses)

print("Total Travel Expense:", total_expense)

if total_expense <= budget:
    print("Travel is within budget")
else:
    print("Travel budget exceeded")

largest = 0

for i in range(1, len(expenses)):
    if expenses[i] > expenses[largest]:
        largest = i

print("Largest Expense Category:", categories[largest])
```

8. Test Cases

Test Case	Budget	Total Expenses	Expected Result
1	■50,000	■32,130	Within budget
2	■20,000	■32,130	Budget exceeded
3	■40,000	■32,130	Within budget

9. Sample Output

```
Enter travel budget in INR: 50000

Hotel
Enter foreign currency amount: 200
Enter exchange rate: 90
Expense in INR: 18360.0

Food
Enter foreign currency amount: 100
Enter exchange rate: 90
Expense in INR: 9180.0

Transport
Enter foreign currency amount: 50
Enter exchange rate: 90
Expense in INR: 4590.0

Total Travel Expense: 32130.0
Travel is within budget
Largest Expense Category: Hotel
```

10. Conclusion

The program converts foreign expenses into Indian rupees and calculates total travel spending. It also identifies the largest expense category and checks whether the spending is within the budget.

CASE STUDY 10: CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

To develop a Python program to calculate the financial requirements of a campus event. The program considers venue, equipment, catering, transport, certificates, and sponsorship. It calculates participant contributions and determines the minimum registrations required to break even.

2. Objectives

• To calculate the total event cost.

• To calculate fixed and variable expenses.

• To identify sponsorship shortfalls.

• To calculate the participant contribution.

• To calculate the minimum registrations needed to break even.

3. Assumptions

• Venue, equipment, and transport costs are fixed costs.

• Catering and certificate costs depend on the number of participants.

• Sponsorship is a fixed amount.

• Each participant pays a fixed registration fee.

• Break-even occurs when total income equals or exceeds total event cost.

4. Algorithm

Start the program.

Read venue, equipment, and transport costs.

Read catering and certificate costs per participant.

Read sponsorship and registration fee.

Calculate total fixed costs.

Calculate the variable cost per participant.

Calculate the total event cost for the expected participants.

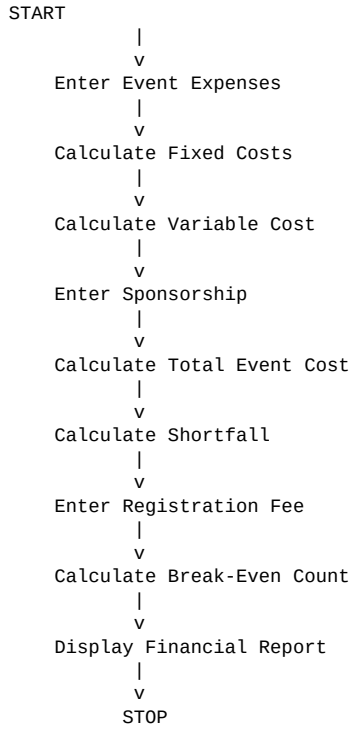
Calculate the sponsorship shortfall.

Calculate the minimum registrations needed to break even.

Display the financial report.

Stop the program.

5. Flowchart



6. Pseudocode

```
START
INPUT venue cost
INPUT equipment cost
INPUT transport cost
INPUT catering cost per person
INPUT certificate cost per person
INPUT sponsorship
INPUT registration fee

fixed_cost = venue + equipment + transport

person_cost = catering + certificate

INPUT expected participants

total_cost = fixed_cost + person_cost * participants

shortfall = total_cost - sponsorship

IF shortfall > 0
    DISPLAY shortfall
ELSE
    DISPLAY "No shortfall"
END IF

IF registration fee > person_cost
    remaining = fixed_cost - sponsorship

    IF remaining <= 0
        minimum registrations = 0
    ELSE
        CALCULATE minimum registrations
    END IF
ELSE
    DISPLAY "Registration fee is too low"
END IF

DISPLAY financial report
STOP
```

7. Python Source Code

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
transport = float(input("Enter transport cost: "))

catering = float(input("Enter catering cost per person: "))
certificate = float(input("Enter certificate cost per person: "))

sponsorship = float(input("Enter sponsorship amount: "))
registration = float(input("Enter registration fee per person: "))
participants = int(input("Enter expected participants: "))

fixed_cost = venue + equipment + transport
person_cost = catering + certificate

total_cost = fixed_cost + person_cost * participants
total_income = sponsorship + registration * participants

shortfall = total_cost - sponsorship

print("Total Event Cost:", total_cost)
print("Total Income:", total_income)

if shortfall > 0:
    print("Sponsorship Shortfall:", shortfall)
else:
    print("No Sponsorship Shortfall")

if registration > person_cost:
    remaining = fixed_cost - sponsorship

    if remaining <= 0:
        minimum = 0
    else:
        minimum = int(remaining / (registration - person_cost)) + 1

    print("Minimum Registrations Needed:", minimum)
else:
    print("Registration fee is too low to cover variable costs")
```

8. Test Cases

Test Case	Fixed Cost	Variable Cost per Person	Sponsorship	Registration Fee	Expected Result
1	■18,000	■250	■20,000	■500	Break-even possible
2	■30,000	■300	■10,000	■200	Registration fee too low
3	■15,000	■200	■20,000	■400	Sponsorship covers fixed costs

9. Sample Output

```
Enter venue cost: 10000
Enter equipment cost: 5000
Enter transport cost: 3000
Enter catering cost per person: 200
Enter certificate cost per person: 50
Enter sponsorship amount: 20000
Enter registration fee per person: 500
Enter expected participants: 100

Total Event Cost: 43000.0
Total Income: 70000.0
Sponsorship Shortfall: 23000.0
Minimum Registrations Needed: 0
```

10. Conclusion

The program calculates campus event expenses, sponsorship shortfalls, and the minimum registrations required to break even. It demonstrates the practical use of arithmetic operators, variables, and conditional statements.

Final note: These ten case studies follow the requested assignment structure and use beginner-level Python. The flowcharts are presented in simple text form so they can be copied into a notebook or redrawn using standard flowchart

symbols.